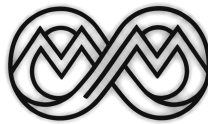


MLOPS ENGINEERING · SESSION 2 OF 5

Shipping to Production

MLflow · W&B · DVC · Code quality · Terraform · GitHub Actions

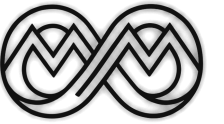
Aya · MLOps MENA Community



Seven topics, one automated pipeline

2.5–3 hours · builds on your Session 1 containerized service

01	MLflow	Experiment tracking, model registry, run comparison
02	Weights & Biases	Cloud-native alternative, sweeps, artifacts
03	DVC	Git for datasets and model files, reproducible pipelines
04	Code quality	Ruff, Black, pre-commit — stop bad code at the door
05	Terraform	Infrastructure as code — provision cloud resources reproducibly
06	GitHub Actions	CI/CD: lint → test → build Docker → push to registry
07	Integration testing	Beyond unit tests — the full request/response cycle



What you built last session

Today we automate everything around it

01

Python package

src/ layout, pyproject.toml

02

Litestar API

/predict, /health, Pydantic

03

Docker

Multi-stage + docker-compose

04

pytest suite

≥ 80% coverage, TestClient

05

Structured logs

JSON + correlation IDs

06

README

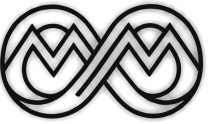
Three-command setup

Session 2 adds experiment tracking, data versioning, infrastructure as code, full CI/CD and integration tests.

TOPIC 01

MLflow

Never lose a good experiment again



model_v2_final_REAL.pkl

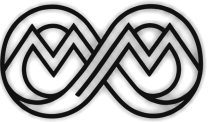
What experiment tracking replaces

WITHOUT MLFLOW

- model_v1_final.pkl
- model_v1_final_FINAL.pkl
- model_v2_fixed_real.pkl
- Which one scored 0.94 again?
- What hyperparameters did that run use?

WITH MLFLOW

- **Experiment** — a named group of related runs
- **Run** — one training execution with its params and metrics
- **Artifact** — files attached to a run: model, plots, data
- **Registry** — versioned model store with a Staging → Production lifecycle



Pointing your code at a tracking server

Two lines before any training code runs

```
train.py

1 # start the server (already in your docker-compose.yml)
2 #   docker compose up -d mlflow
3 #   then open http://localhost:5000
4
5 import mlflow
6
7 mlflow.set_tracking_uri("http://localhost:5000")
8 mlflow.set_experiment("ride-duration-model")
9
10 # every run from now on lands in that experiment
11 # nothing else in your training code has to change
```

Tracking URI

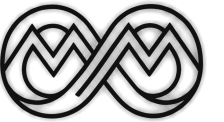
Where runs are written. Point it at localhost in development and at the team server in CI.

Experiment

A named bucket of runs. One experiment per problem, not per attempt.

Already containerized

MLflow runs as a service beside your API — the same docker-compose file you wrote last session.



Logging params, metrics and models

Every training run fully captured, in code

train.py

```
1 import mlflow, mlflow.sklearn
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.metrics import mean_absolute_error
4
5 with mlflow.start_run(run_name="rf-baseline") as run:
6
7     params = {"n_estimators": 100, "max_depth": 6}
8     mlflow.log_params(params)
9
10    model = RandomForestRegressor(**params)
11    model.fit(X_train, y_train)
12
13    mae = mean_absolute_error(y_val, model.predict(X_val))
14    mlflow.log_metric("mae", mae)
15
16    mlflow.sklearn.log_model(
17        model, artifact_path="model",
18        registered_model_name="RideDurationModel")
```

start_run()

A context manager — the run closes even if training crashes halfway.

log_params()

A dict of hyperparameters. Searchable and comparable across every run in the UI.

log_metric()

One number tied to this run. Log it repeatedly to get a training curve.

log_model()

Saves weights plus metadata. registered_model_name puts it straight into the Registry.

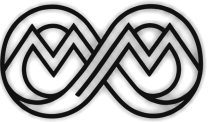


This is why MLflow exists

Three model families, twelve runs, one sortable table

RUN NAME	MODEL	CONFIG	MAE (MIN)	DURATION
lr-baseline	Linear Reg	—	8.42	2s
xgb-shallow	XGBoost	100 · depth 3	5.84	8s
xgb-medium	XGBoost	200 · depth 6	5.12	14s
xgb-tuned	XGBoost	250 · lr 0.05	4.87	31s
mlp-medium	PyTorch MLP	[128, 64]	5.44	78s
mlp-large	PyTorch MLP	[256, 128]	4.92	142s
mlp-dropout	PyTorch MLP	dropout 0.3	4.63	158s
★ xgb-best	XGBoost	300 · lr 0.03	4.41	48s

Without tracking you would re-read twelve scripts to answer this. With it, you sort one column.



Promote models through stages

None → Staging → Production → Archived

registry.py

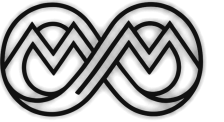
```
1 from mlflow import MlflowClient
2
3 client = MlflowClient("http://localhost:5000")
4
5 # list every registered version
6 for v in client.search_model_versions(
7     "name='RideDurationModel'"):
8     print(v.version, v.current_stage)
9
10 # promote version 3 to Staging
11 client.transition_model_version_stage(
12     name="RideDurationModel",
13     version="3", stage="Staging")
14
15 # serving code loads by stage, never by number
16 model = mlflow.sklearn.load_model(
17     "models:/RideDurationModel/Production")
```

None Just registered

Staging Being validated

Production Serving live traffic

Archived Retired



Six things MLflow does

Most teams only ever discover half of them

01

Experiment tracking

`log_params()`, `log_metric()`, `log_artifact()`.
Compare any two runs side by side in the UI.

02

Autologging

One line captures params, metrics, model signature and feature importance. sklearn, PyTorch, XGBoost, HuggingFace.

03

Artifact logging

Confusion matrices, feature importance plots, datasets, SHAP charts — the full run story, not just weights.

04

Model registry

Versioned store with a lifecycle. Load by stage name so serving code never hardcodes a version.

05

Run comparison UI

Select runs, get a table plus a parallel-coordinates chart. Sort by any metric.

06

Tags and search

Tag runs with team, dataset version, git commit. Search across all runs with SQL-like syntax.



MLflow for LLM and RAG work

Same workflow — only the experiment is different

CLASSICAL ML	LLM / RAG EQUIVALENT	WHY IT MATTERS
Hyperparameters (lr, batch size, n_estimators)	Prompt version, chunk_size, chunk_overlap, temperature	A different prompt is a different model. Chunk size is retrieval quality.
Training metrics (MAE, F1, accuracy)	RAGAS faithfulness, answer relevancy, context recall	These are your eval metrics. There is no training loop to watch.
Model artifact (model.pkl, model.onnx)	System prompt text, embedding model, RAG config bundle	The prompt is the artifact. Version it like a model.
Model registry (None → Staging → Production)	Prompt registry (Draft → Staging → Production)	Serving loads the prompt by stage, exactly as it loads a model.
Run comparison (which XGBoost beat the MLP?)	Prompt A vs prompt B vs Mistral 7B vs Llama 3.1	Identical workflow — sort by your primary RAGAS metric.

MLflow 2.12 and later ship a prompt registry: `register_prompt()` and `set_prompt_alias()`.

TOPIC 02

Weights & Biases

The cloud-native alternative

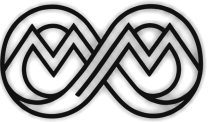


Same job, different infrastructure

Pick on hosting and team shape, not on features

	MLFLOW	WEIGHTS & BIASES
Hosting	Self-hosted on your server	Cloud SaaS (wandb.ai)
Setup effort	~10 min with docker-compose	pip install + wandb login
Hyperparameter search	Manual, or an Optuna plugin	Built-in sweeps: Bayesian, grid, random
Collaboration	A shared MLflow server	Projects, teams, reports
Artifact storage	Local or S3	W&B cloud artifact store
Cost	Free and open-source	Free tier, paid for teams
Best for	On-prem, enterprise, Kubernetes	Fast iteration, research, reporting

In this course: learn MLflow deeply. Know W&B for research and team reporting.



Logging a run to the cloud

Same training code, different infrastructure

```
train_wandb.py

1 import wandb
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.metrics import mean_absolute_error
4
5 run = wandb.init(project="ride-duration",
6                 name="rf-baseline")
7
8 params = {"n_estimators": 100, "max_depth": 6}
9 wandb.config.update(params)
10
11 model = RandomForestRegressor(**params)
12 model.fit(X_train, y_train)
13 mae = mean_absolute_error(y_val, model.predict(X_val))
14
15 wandb.log({"mae": mae, "val_size": len(y_val)})
16
17 artifact = wandb.Artifact("ride-model", type="model")
18 artifact.add_file("model.pkl")
19 run.log_artifact(artifact); run.finish()
```

wandb.init()

Creates the run on wandb.ai and returns a run object you keep for artifacts.

wandb.config

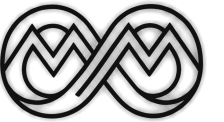
The immutable config for this run: hyperparameters, model type, dataset path.

wandb.log()

Any metric, at any step. Call it inside a training loop and you get live curves.

Artifact

A versioned file store. Datasets, models and eval reports become first-class objects.



Hyperparameter search, built in

Define a space, let the agent search it

```
sweep.py

1 sweep_config = {
2     "method": "bayes",
3     "metric": {"name": "mae", "goal": "minimize"},
4     "parameters": {
5         "n_estimators": {"values": [50, 100, 200]},
6         "max_depth": {"min": 3, "max": 10},
7     },
8 }
9
10 sweep_id = wandb.sweep(sweep_config,
11                        project="ride-duration")
12
13 # run 20 trials, W&B picks the next config each time
14 wandb.agent(sweep_id, function=train_fn, count=20)
```

method: bayes

Bayesian search uses previous trials to pick the next config. Far fewer runs than grid search.

The agent

A worker process that pulls a config, trains, reports the metric, repeats.

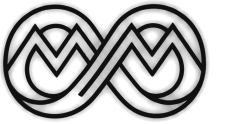
Where MLflow differs

MLflow needs Optuna or a manual loop for the same job.

TOPIC 03

DVC

Git for datasets and models



Git cannot hold your data

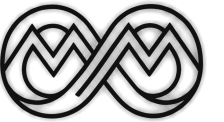
It tracks text. Your dataset is 500MB of it

GIT ALONE

- data/raw/rides.csv — 500 MB
- models/rf_v1.pkl — 200 MB
- models/rf_v2.pkl — 200 MB
- Git refuses files over 100 MB
- History becomes unusable long before that

DVC STORES THE POINTER

- data/raw/rides.csv.dvc — a few hundred bytes
- models/rf_v1.pkl.dvc — a few hundred bytes
- The real files live in S3, GCS or a local cache
- Git tracks the pointers, DVC tracks the data
- Clone the repo, run dvc pull, get the exact dataset



Tracking your first dataset

From git init to data in S3, in five commands

terminal

```
1 # 1. initialise DVC inside your git repo
2 git init && dvc init
3 git add .dvc/ && git commit -m "init: add dvc"
4
5 # 2. track a large file
6 dvc add data/raw/rides.csv
7 #   creates data/raw/rides.csv.dvc (tiny pointer)
8 #   adds the real file to .gitignore automatically
9 git add data/raw/rides.csv.dvc data/raw/.gitignore
10
11 # 3. define a remote
12 dvc remote add -d s3remote s3://mlops-dvc-artifacts/data
13 dvc remote modify s3remote region us-east-1
14
15 # 4. push data, then push code
16 dvc push && git push
17
18 # 5. on any other machine
19 git clone <repo> && dvc pull
```

dvc add

Copies the file into .dvc/cache and writes a small pointer. The original is gitignored.

The .dvc file

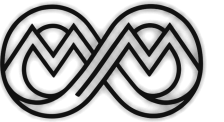
Plain YAML with an md5 hash and a size. This is what goes into Git.

-d means default

DVC uses that remote for push and pull without naming it every time.

Content-addressed

The same file is uploaded once, whatever branch or tag references it.



Reproducible pipelines

dvc repro re-runs only what actually changed

dvc.yaml

```
1 stages:
2   prepare:
3     cmd: python src/prepare.py
4     deps:
5       - src/prepare.py
6       - data/raw/rides.csv
7     outs:
8       - data/processed/train.parquet
9
10  train:
11    cmd: python src/train.py
12    deps:
13      - src/train.py
14      - data/processed/train.parquet
15    params:
16      - configs/config.yaml:
17        - model.n_estimators
18    outs:
19      - models/rf_model.pkl
```

deps

Inputs. If any of them change, DVC re-runs this stage on the next dvc repro.

params

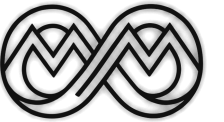
Hyperparameters pulled from config.yaml. DVC notices when a value changes.

outs

Outputs DVC caches. Untouched deps mean the stage is skipped entirely.

The habit

Commit dvc.yaml and dvc.lock. Anyone can rebuild your pipeline from those two files.



Data version tied to code version

One checkout restores any historical experiment

terminal

```
1  # v1.0 — baseline model, 500k rows
2  git tag -a v1.0 -m "baseline model"
3  git push origin v1.0
4
5  # dataset grows to 1M rows
6  dvc add data/raw/rides.csv      # new hash in the .dvc file
7  git commit -am "data: expand to 1M rows"
8  git tag -a v2.0 -m "retrained on 1M rows"
9  git push origin v2.0 && dvc push
10
11 # switch between data versions instantly
12 git checkout v1.0 && dvc checkout
13 git checkout v2.0 && dvc checkout
14
15 # inspect
16 dvc status                      # what is out of sync?
17 dvc diff v1.0 v2.0             # what data changed between tags?
```

git tag + dvc checkout

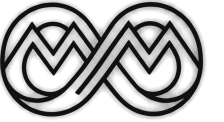
Git restores the code and pointers. DVC swaps the actual files to match.

dvc status

Like git status, for data. Shows modified, missing and unpushed files.

dvc diff

Old and new hashes for every changed file between two tags. Auditable data lineage.



Pulling data inside the pipeline

Credentials as secrets, never in the repo

```
.github/workflows/train.yml

1 - name: Configure DVC remote credentials
2   env:
3     AWS_ACCESS_KEY_ID:    ${ secrets.AWS_ACCESS_KEY_ID }
4     AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
5   run: |
6     pip install "dvc[s3]"
7     dvc pull data/processed/  # only what training needs
8
9 - name: Run the DVC pipeline
10  run: dvc repro              # re-runs only changed stages
11
12 # .dvc/config is committed to git and holds no secrets:
13 #   [core]
14 #     remote = myremote
15 #     url = s3://mlops-dvc-artifacts
```

Partial pull

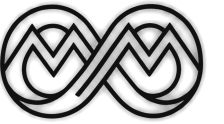
Pull only the folder training needs. CI runners do not want your whole 40GB remote.

Secrets, not config

.dvc/config holds the bucket URL. The credentials come from GitHub secrets at runtime.

Multiple remotes

A local cache on SSD for dev, a staging bucket for the team, a locked-down production bucket.



DVC and MLflow together

One model version, one dataset commit, reproducible forever

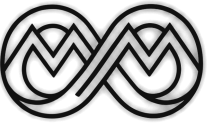
DVC VERSIONS THE DATA

- Datasets, features, preprocessed outputs
- Stored in S3, GCS or a local remote
- Referenced by a git commit and tag
- `dvc tag rides-v2.1 → abc123f`

MLFLOW VERSIONS THE MODEL

- Model weights, parameters, metrics
- Stored on the tracking server
- Referenced by `run_id`
- `run_id = 9a3f8b2c...`

```
model v3.2 ← run_id 9a3f8b2c ← dvc tag rides-v2.1 ← git commit abc123f ← 2.1M rows, Jan 2024
```



DVC for LLM and RAG work

Version the knowledge base, the chunks and the eval set

01

Knowledge base

Classically the training dataset. Here it is the document corpus — legal PDFs, regulation texts, contract templates.

```
dvc add data/knowledge_base/
```

02

Chunk store

Classically the feature engineering output. Here it is chunks.jsonl, after splitting, cleaning and metadata injection.

```
dvc repro chunk_docs
```

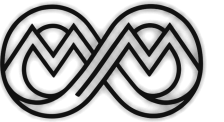
03

Eval set

Classically the test labels. Here it is 50–100 question, reference answer and relevant document triples.

```
dvc add  
data/eval/test_set.jsonl
```

A new law is passed: add the document, dvc add, commit, and dvc repro rebuilds the index.



Versioning beyond files

What else to version, and what to use when DVC does not fit

ALSO WORTH VERSIONING

- **The vector index** — chunks plus embedding model plus index params
- **Embedding model version** — v1 and v2 are different vector spaces
- **Fine-tuning data** — instruction pairs, preference pairs, LoRA adapters
- **Tool schemas and guardrails** — a changed signature changes behaviour
- **The judge config** — the judge model and its grading prompt are part of the metric

WHEN DVC DOES NOT FIT

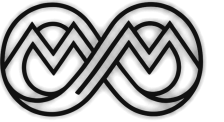
- **Vector DB snapshots** — build docs_v3, then swap the alias atomically
- **Database PITR** — pg_dump or an RDS snapshot tagged with the git SHA
- **Table time travel** — Delta Lake or Iceberg VERSION AS OF
- **S3 object versioning** — plus a manifest of object version IDs
- **lakeFS or Nessie** — git-like branching over the whole data lake

DVC for files you can enumerate. Snapshots for anything a live system keeps writing.

TOPIC 04

Code quality

Make bad code impossible to commit



Three tools that end style debates

Install once, and bad code never reaches the branch

01

Ruff

An ultra-fast Python linter written in Rust. Replaces Flake8, isort and pylint at 10–100× the speed.

```
ruff check src/
```

02

Black

The opinionated formatter. Zero configuration — every file in the repo looks identical.

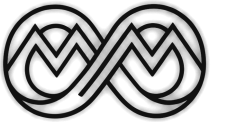
```
black src/
```

03

pre-commit

A git hook framework. Runs Ruff and Black before every commit, automatically.

```
pre-commit install
```



The config file, in full

Drop it in the repo root and install the hook

```
.pre-commit-config.yaml

1 repos:
2   - repo: https://github.com/astral-sh/ruff-pre-commit
3     rev: v0.4.4
4     hooks:
5       - id: ruff                # linter, auto-fixes safe issues
6         args: [--fix]
7       - id: ruff-format        # formatter, replaces Black
8
9   - repo: https://github.com/pre-commit/pre-commit-hooks
10     rev: v4.6.0
11     hooks:
12       - id: trailing-whitespace
13       - id: end-of-file-fixer
14       - id: check-yaml
15       - id: check-merge-conflict
```

rev is a version pin

Pin the hook version so the whole team formats identically. Bump it deliberately.

args: [--fix]

Ruff repairs what it safely can. What it cannot repair, it blocks the commit on.

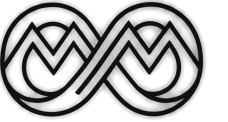
The small hooks matter

Trailing whitespace and unresolved merge conflicts cause more review noise than real bugs.

TOPIC 05

Terraform

Infrastructure as code



Write code, get infrastructure

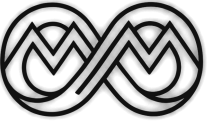
Describe the infrastructure you want; Terraform makes it real

WITHOUT TERRAFORM

- Click through the AWS console by hand
- No record of what you did, or why
- You cannot recreate the same environment
- Every teammate sets it up slightly differently
- Disaster recovery means starting from scratch

WITH TERRAFORM

- The infrastructure lives in .tf files
- Git-tracked, with full history and review
- terraform apply rebuilds everything
- Same code, same infrastructure, every time
- Disaster recovery is git clone plus apply



Four words you need

Provider, resource, state, plan and apply

01

Provider

The plugin that talks to a cloud — AWS, GCP, Azure, Docker.
Downloaded by terraform init.

02

Resource

One piece of infrastructure: an S3 bucket, an EC2 instance, a DNS record.

03

State

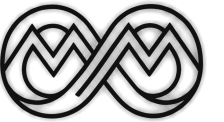
terraform.tfstate — the map between your code and what actually exists
in the cloud.

04

Plan and apply

plan previews the change, apply performs it. Always plan first, and read
the output.

Never commit terraform.tfstate. It contains secrets — use an S3 backend for team state.



Six pieces of infrastructure

Everything this stack needs in order to run

01

S3 bucket

DVC remote storage

02

MLflow server

Tracking and registry

03

Docker registry

CI/CD image pushes

04

Secrets manager

API keys and credentials

05

Compute VM

Training jobs

06

IAM roles

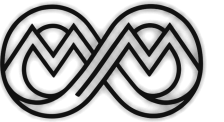
Who can do what

WITHOUT IAC

- New joiner needs a two-hour onboarding call
- Staging bucket deleted, and no record of its config
- Cost audit — nobody knows what is running, or why

WITH IAC

- git clone, terraform apply, five minutes, identical env
- git revert, terraform apply, restored in two minutes
- terraform show lists every resource and its owner



Start from the Registry

registry.terraform.io gives you the blocks to copy

```
infra/main.tf

1  # copied from registry.terraform.io → Providers → AWS
2  terraform {
3    required_providers {
4      aws = {
5        source  = "hashicorp/aws"
6        version = "~> 5.0"      # pin the major version
7      }
8    }
9  }
10
11 provider "aws" {
12   region = "us-east-1"        # the only thing you change
13 }
14
15 # copied from the aws_s3_bucket documentation page
16 resource "aws_s3_bucket" "dvc" {
17   bucket = "mlops-dvc-artifacts-ride-duration"
18 }
```

The Registry is the docs

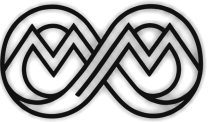
Every resource has a page with the exact block to copy. Search, copy, fill in your values.

Pin the version

~> 5.0 accepts 5.x but never 6.0. Unpinned providers break builds months later.

You still need credentials

Terraform uses your AWS CLI profile or environment variables. It does not create the account.



Variables and the seven commands

Parameterise once, then reuse everywhere

terminal

```
1  # infra/variables.tf
2  #   variable "project_name" {
3  #       type      = string
4  #       default = "ride-duration"
5  #   }
6  # infra/terraform.tfvars (gitignored, holds real values)
7
8  terraform init           # providers + backend
9  terraform fmt            # auto-format .tf files
10 terraform validate       # syntax check, no cloud calls
11 terraform plan -out=tfplan # preview, read-only
12 terraform apply tfplan    # create or update
13 terraform output ecr_repo_url # read a value back out
14 terraform destroy         # tear it all down
```

Never commit .tfstate

It holds secrets. Gitignore it and use an S3 backend so the team shares one state.

tfvars stays local

Account IDs and bucket names are real values. Keep them out of git; use CI secrets.

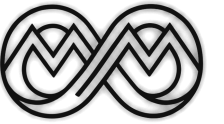
Modules

Package a VPC or a cluster once, then reuse it across staging and production.

TOPIC 06

CI/CD with GitHub Actions

Automate everything on every push



Continuous integration and delivery

Automating the verification and packaging of every code change

CI – CONTINUOUS INTEGRATION

- Lint and format check
- Unit and integration tests
- Coverage gate at 80%
- Build verification

CD – CONTINUOUS DELIVERY

- Docker image built
- Image pushed to the registry
- Artifact versioned by git SHA
- Optionally, automatic deploy

Trigger

A push, a PR, a schedule,
a manual click

Runner

A clean VM that executes
your steps

Steps

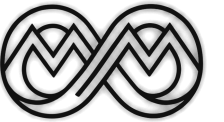
Install, test, build, push

Secrets

Encrypted credentials
injected at runtime

Artifacts

Coverage reports,
images, wheels



What starts a workflow

Events, branches, schedules and manual buttons

```
.github/workflows/ci.yml

1  on:
2    push:
3      branches: [main]           # only pushes to main
4
5    pull_request:
6      branches: [main]
7      paths:
8        - "src/**"              # only when src/ changes
9
10   schedule:
11     - cron: "0 2 * * 1"        # Mondays at 02:00 UTC
12
13   workflow_dispatch:           # a manual button in the UI
14     inputs:
15       reason:
16         description: "Why are you triggering this?"
17         required: true
```

paths filters

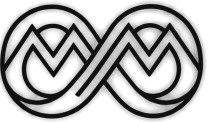
Do not run the full pipeline because someone fixed a typo in the README.

schedule

Cron in UTC. This is how a weekly retrain gets kicked off without a human.

workflow_dispatch

Adds a Run workflow button, with typed inputs. Your emergency lever.



A real CI job, top to bottom

Checkout, Python, cache, install, lint, test

.github/workflows/ci.yml

```
1 jobs:
2   lint-and-test:
3     runs-on: ubuntu-latest
4     steps:
5       - uses: actions/checkout@v4
6
7       - uses: actions/setup-python@v5
8         with:
9           python-version: "3.11"
10
11      - uses: actions/cache@v4
12        with:
13          path: ~/.cache/pip
14          key: ${runner.os}-pip
15
16      - run: pip install -e .[dev]
17      - run: ruff check src/ tests/
18      - run: pytest --cov=src --cov-fail-under=80 -v
```

runs-on

A fresh VM per run. Free for public repos; 2000 minutes a month on the free private tier.

actions/cache

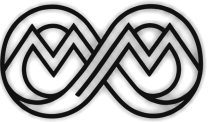
Caches pip between runs, keyed on the hash of pyproject.toml. Invalidated when deps change.

--cov-fail-under=80

The quality gate. CI fails if coverage drops, so it cannot drift downward quietly.

uses vs run

uses pulls a prebuilt action from the Marketplace. run executes raw shell.



Building the image and pushing it

lint → test → build → push, in one file

.github/workflows/ci.yml

```
1  build-and-push:
2    runs-on: ubuntu-latest
3    needs: lint-and-test          # only if tests passed
4    if: github.ref == "refs/heads/main"
5    steps:
6      - uses: actions/checkout@v4
7
8      - uses: docker/login-action@v3
9        with:
10          username: ${ secrets.DOCKERHUB_USERNAME }
11          password: ${ secrets.DOCKERHUB_TOKEN }
12
13      - uses: docker/metadata-action@v5
14        id: meta
15        with:
16          images: ${ secrets.DOCKERHUB_USERNAME }/ride-api
17          tags: |
18            type=sha,prefix=,format=short
19            type=raw,value=latest
```

needs:

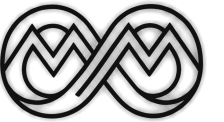
A job dependency. The image is only built if the test job passed — broken images never ship.

metadata-action

Generates two tags automatically: the short git SHA and :latest. Both get pushed.

Layer cache

cache-from the previous :latest cuts a three-minute build to under thirty seconds.



Testing many versions, handling secrets

Three Python versions in parallel, credentials injected

```
.github/workflows/ci.yml

1  lint-and-test:
2    strategy:
3      matrix:
4        python-version: ["3.10", "3.11", "3.12"]
5        fail-fast: false      # one failure, others continue
6    steps:
7      - uses: actions/setup-python@v5
8        with:
9          python-version: ${ matrix.python-version }
10     - run: pytest -v        # three parallel jobs
11
12  deploy:
13    environment: production  # environment-scoped secrets
14    steps:
15      - name: Deploy
16        env:
17          REGISTRY_TOKEN: ${ secrets.MODEL_REGISTRY_TOKEN }
18      run: python scripts/deploy.py
```

matrix

Spawns one job per value. Three Python versions means three concurrent runners.

fail-fast: false

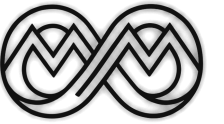
You see the whole compatibility picture, not just the first failure.

Environments

Different secrets for staging and production, plus an optional manual approval gate.

Never echo a secret

GitHub redacts them in logs. Do not put one in a run: command to find out.



Four ways to start a training run

A software pipeline knows one of these. An ML pipeline needs all four

01 Git push or PR

Developer changes model code, features or training logic

```
ci.yml

1 on:
2   push:
3     branches: [main]
4   pull_request:
5     paths:
6       - "src/**"
7       - "train.py"
```

Fastest feedback loop. Best for code changes.

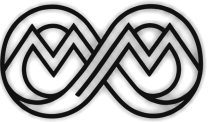
02 Scheduled retrain

The model needs fresh data periodically, regardless of code

```
ci.yml

1 on:
2   schedule:
3     - cron: "0 2 * * 1"
4 jobs:
5   retrain:
6     steps:
7       - run: dvc pull
8       - run: python train.py
```

Best for time-sensitive models: fraud, pricing, news.



Four ways to start a training run

Continued — the two that only exist in ML systems

03 Drift signal

Monitoring detects PSI above 0.25 or a degrading MAE

```
ci.yml

1 on:
2   repository_dispatch:
3     types: [drift-detected]
4 jobs:
5   emergency-retrain:
6     if: github.event.client_payload.psi > 0.25
```

Best for production models with active drift monitoring.

04 Manual or ad-hoc

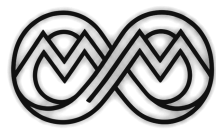
Regulatory change, known data corruption, emergency correction

```
ci.yml

1 on:
2   workflow_dispatch:
3     inputs:
4       reason:
5         required: true
6       data_version:
7         default: "latest"
```

Best for urgent corrections that need human sign-off.

Most teams use all four. Triggers 02–04 are what separate an ML pipeline from a software pipeline.



What one git push sets off

Seven stages, four to six minutes, no human in the loop

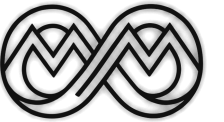
01	<code>git push</code>	Developer pushes to main
02	<code>Ruff + Black</code>	Lint and format, fail fast on style
03	<code>pytest --cov</code>	Unit and integration tests, $\geq 80\%$ coverage
04	<code>docker build</code>	Multi-stage build from your Dockerfile
05	<code>docker push</code>	Tagged with the git SHA, pushed to the registry
06	<code>terraform apply</code>	Infrastructure drift check, applied if changed
07	Deploy	Pull the image, restart the service, health check

If any stage fails the chain stops immediately and GitHub notifies you. Nothing broken reaches production.

TOPIC 07

Integration testing

Beyond unit tests — the whole request cycle



Testing the whole request cycle

Every route, every status code, every edge case

tests/test_integration.py

```
1 import pytest
2 from litestar.testing import TestClient
3 from unittest.mock import MagicMock
4 from src.api import app
5
6 @pytest.fixture(scope="module")
7 def client():
8     mock = MagicMock(spec=RideDurationModel)
9     mock.predict.return_value = 15.5
10    app.dependency_overrides[RideDurationModel] = lambda: mock
11    with TestClient(app=app) as c:
12        yield c
13
14 def test_predict_success(client):
15     r = client.post("/predict",
16                     json={"distance_km": 5.0, "passengers":
17 })
18     assert r.status_code == 201
19     assert "duration_min" in r.json()
```

scope="module"

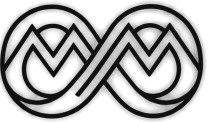
The fixture is built once per file, not once per test. No re-mocking twenty times.

dependency_overrides

Swap the real model for a mock. You are testing the API, not the model.

Real routing

Middleware, dependency injection and serialization all run — that is what makes it integration.



Bad input and broken contracts

Four rejection cases in one test function

tests/test_integration.py

```
1  @pytest.mark.parametrize("payload,expected", [
2      ({"distance_km": -1.0, "passengers": 2}, 422), # negative
3      ({"distance_km": 5.0, "passengers": 0}, 422), # zero
4      ({"passengers": 2}, 422), # missing
5      ({}, 422), # empty
6  ])
7  def test_predict_validation(client, payload, expected):
8      assert client.post("/predict",
9                          json=payload).status_code == expected
10
11 def test_health(client):
12     assert client.get("/health").json()["status"] == "healthy"
13
14 def test_response_schema(client):
15     body = client.post("/predict",
16                       json={"distance_km": 5.0}).json()
17     assert set(body.keys()) == {"duration_min", "status"}
```

parametrize

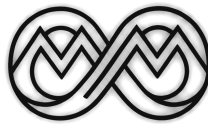
One function, four cases. Every validation rule gets its own assertion and failure message.

test_health

Load balancers ping /health. A broken health endpoint kills deployments silently.

Contract test

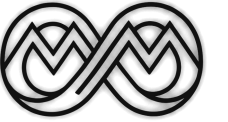
Comparing the key set catches any added or removed field — the classic breaking change.



The automated ML pipeline

Seven additions to the service you already shipped

01	MLflow tracking	At least three logged runs with params, MAE and a model artifact. Best run promoted to Staging.
02	W&B side by side	The same experiment logged to both. Compare the two UIs in your README.
03	DVC pipeline	dvc.yaml with prepare → train → evaluate. dvc repro runs clean. Data lives in S3.
04	Code quality	Ruff and pre-commit installed. Every file passes ruff check.
05	Terraform	S3 bucket provisioned through Terraform. terraform plan output committed as an artifact.
06	GitHub Actions	Full CI: lint, test, docker build, push to Docker Hub. Green badge in the README.
07	Integration tests	/predict, /health, all 422 validation cases, and the response schema contract.



The homework and Next live session

-
- 01 DVC pipelines and yaml files
 - 02 Ruff capabilities
 - 03 Terraform as optional as a very advanced topic
 - 04 Integration testing
 - 05 Terraform
 - 06 W&B , a quick experiment
-

Next Live session, CI/CD with (Eng Abdallah ELmalawany)

Live session, Next Saturday 29 August

Register at Zomra: <https://zomra.io/free-sessions/4f2b8aa6-1018-44b2-a151-a35a63d93bbc>



DVC and Git

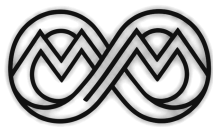
The commands you will use every week

EVERYDAY

dvc init	Initialise DVC in a git repo
dvc add <file>	Track a data file, creating a .dvc pointer
dvc push	Upload tracked data to the remote
dvc pull	Download tracked data from the remote
dvc repro	Re-run the pipeline stages that changed
dvc checkout	Swap data files to match the current commit

INSPECTION

dvc status	What data is out of sync with the remote
dvc diff v1.0 v2.0	Data changes between two tags
dvc params diff	Parameter changes between commits
dvc metrics show	Every tracked metric
dvc dag	Print the pipeline graph
dvc gc --all-commits	Delete cache not referenced by any commit



GitHub Actions and Terraform

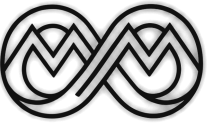
CLI commands worth memorising

GITHUB CLI

gh workflow list	List every workflow in the repo
gh run list	List recent workflow runs
gh run view <id>	Details of one run
gh run rerun <id>	Re-run a failed workflow
gh secret set NAME	Set a repository secret from the CLI
act	Run Actions locally, before pushing

TERRAFORM

terraform init	Download providers, initialise the backend
terraform fmt	Auto-format every .tf file
terraform validate	Syntax check, no cloud calls
terraform plan -out=tfplan	Preview and save the plan
terraform apply tfplan	Apply the saved plan
terraform destroy	Destroy every managed resource



Where to go next

Everything on this slide is free

MLflow docs

mlflow.org/docs/latest

W&B docs

docs.wandb.ai

DVC

dvc.org – interactive tutorials

Ruff

docs.astral.sh/ruff

GitHub Actions

docs.github.com/en/actions

Terraform

developer.hashicorp.com/terraform/tutorials

MLOps Zoomcamp

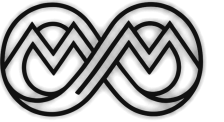
github.com/DataTalksClub/mlops-zoomcamp

Made With ML

madewithml.com

MLOps Community

mlops.community



Session 3 · Scale, load and reliability

Where the pipeline meets real traffic

01

Airflow

Orchestrate retraining on a schedule or a drift trigger

02

Batch and streaming

Batch scoring on S3, streaming inference on Kinesis and Lambda

03

Locust load testing

Simulate 100+ concurrent users, find the p95 and p99 bottlenecks

04

BentoML serving

Micro-batching, adaptive runners, gRPC — beyond raw FastAPI

05

Deployment strategies

Questions?

Session 2 complete. Push it to GitHub.

Aya · MLOps MENA Community

<https://www.linkedin.com/company/mlops-mena> #mlops-mena